LAB 1:

```
import re

import nltk

from nltk.tokenize import word_tokenize

from nltk.corpus import stopwords


nltk.download("punkt_tab", quiet=True)

nltk.download("stopwords", quiet=True)


text = "Hello World! Year 2026"


print("Original Text:")

print(text)

print("-" * 50)


tokens = word_tokenize(text)

print("Tokens:")

print(tokens)

print("-" * 50)


stop_words = set(stopwords.words('english'))

filtered_tokens = []


for word in tokens:


    word = word.lower()

    if word.isalpha() and word not in stop_words:

        filtered_tokens.append(word)


print("Filtered tokens:")

print(filtered_tokens)
```

```
print("-" * 50)


def is_english_script(text):


    pattern = re.compile(r'^[A-Za-z0-9\s.,!?\'-]+$')

    return bool(pattern.match(text))


if is_english_script(text):

    print("Script Validation: Text is English (Latin script)")

else:

    print("Script Validation: Text contains invalid script")
```

LAB2:

```
import nltk

from nltk.tokenize import word_tokenize

from nltk.corpus import stopwords

from nltk.corpus import wordnet

nltk.download('punkt')

nltk.download('stopwords')

from nltk.stem import PorterStemmer

from nltk.stem import WordNetLemmatizer

nltk.download('wordnet')

nltk.download('averaged_perceptron_tagger')

from nltk import pos_tag

text = "I am studying btech at MIT WPU which is a private university located in Pune, India."

print("Original Text:")

print(text)

print()

#tokenization

tokens = word_tokenize(text)

print("Tokens:")

print(tokens)

print()

#pos tagging-noun,verbs,adjective

pos_tags = pos_tag(tokens)

print("POS:")

for word, tag in pos_tags:

    if tag.startswith('N'):

        print(word, "- Noun")

    elif tag.startswith('V'):

        print(word, "- Verb")

    elif tag.startswith('J'):

        print(word, "- Adjective")

print()
```

```
#stopword removal
stop_words = set(stopwords.words('english'))
filtered_words = []
for word in tokens:
    if word.isalpha() and word.lower() not in stop_words:
        filtered_words.append(word.lower())
print("After Stopword Removal:")
print(filtered_words)
print()
#stemming
stemmer = PorterStemmer()
stemmed_words = []
for word in filtered_words:
    stemmed_words.append(stemmer.stem(word))
print("Stemmed Words:")
print(stemmed_words)
print()
def get_wordnet_pos(tag):
    if tag.startswith('J'):
        return wordnet.ADJ
    elif tag.startswith('V'):
        return wordnet.VERB
    elif tag.startswith('N'):
        return wordnet.NOUN
    else:
        return wordnet.NOUN
print("WordNet POS Tags:")
for word, tag in pos_tags:
    if word.isalpha() and word.lower() not in stop_words:
        wn_tag = get_wordnet_pos(tag)

        if wn_tag == wordnet.NOUN:
```

```
        pos_name = "NOUN"
    elif wn_tag == wordnet.VERB:
        pos_name = "VERB"
    elif wn_tag == wordnet.ADJ:
        pos_name = "ADJECTIVE"
    elif wn_tag == wordnet.ADV:
        pos_name = "ADVERB"

    print(word, "->", pos_name)
print()
# lemmatization
lemmatizer = WordNetLemmatizer()
lemmatized_words = []
for word, tag in pos_tags:
   if word.isalpha() and word.lower() not in stop_words:
      lemma = lemmatizer.lemmatize(word.lower(), get_wordnet_pos(tag))
      lemmatized_words.append(lemma)
print("Lemmatized Words:")
print(lemmatized_words)
print()
```

LAB3:

```
from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.feature_extraction.text import CountVectorizer

from sklearn.metrics.pairwise import cosine_similarity

from sklearn.metrics import pairwise_distances

import pandas as pd

d0 = 'This is a sample notebook'

d1 = 'This notebook is sample'

d2 = 'This is another sample notebook'

string = [d0, d1, d2]

TF - IDF

tfidf = TfidfVectorizer()

result = tfidf.fit_transform(string)

print('\nidf values:')

for ele1, ele2 in zip(tfidf.get_feature_names_out(), tfidf.idf_):

    print(ele1, ':', ele2)

print('\nWord indexes:')

print(tfidf.vocabulary_)

print('\ntf-idf value:')

print(result)

print('\ntf-idf values in matrix form:')

print(result.toarray())

count_vectorizer = CountVectorizer()

sparse_matrix = count_vectorizer.fit_transform(string)

doc_term_matrix = sparse_matrix.todense()

df = pd.DataFrame(doc_term_matrix,

                  columns=count_vectorizer.get_feature_names_out(),

                  index=['d0', 'd1', 'd2'])


print("Document-Term Matrix:\n", df)

print("-" * 30)

cosine_sim = cosine_similarity(sparse_matrix)
```

```
sim_df = pd.DataFrame(cosine_sim,
                columns=['d0', 'd1', 'd2'],
                index=['d0', 'd1', 'd2'])


print("Cosine Similarity Matrix:\n", sim_df)
vectorizer = CountVectorizer(binary=True, token_pattern=r"(?u)\b\w+\b")
X = vectorizer.fit_transform(string)
jaccard_distance = pairwise_distances(X.toarray(), metric='jaccard')
jaccard_similarity = 1 - jaccard_distance
df = pd.DataFrame(jaccard_similarity,
            index=['d0', 'd1', 'd2'],
            columns=['d0', 'd1', 'd2'])


print(df)
```

LAB4(finance):

```
import pandas as pd

from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer

from sklearn.metrics import classification_report


def analyze_finance_data():

    analyzer = SentimentIntensityAnalyzer()

    print("Loading Finance Dataset...")

    try:

        df = pd.read_csv('lab5_sentiment_finance.csv', encoding='latin-1').dropna()

    except FileNotFoundError:

        print("Error: lab5_sentiment_finance.csv not found.")

        return


    def get_vader_label(text):

        score = analyzer.polarity_scores(str(text))['compound']

        if score >= 0.05: return "Positive"

        elif score <= -0.05: return "Negative"

        else: return "Neutral"


    print("Applying VADER Sentiment Analysis...")

    df['vader_prediction'] = df['text'].apply(get_vader_label)

    # Capitalize the original category to match VADER's formatting

    df['true_label'] = df['category'].str.capitalize()

    print("\n--- Finance Data Results ---")

    print(df[['text', 'true_label', 'vader_prediction']].head())

    # Generate Classification Report

    print("\n--- Classification Report ---")

    print(classification_report(df['true_label'], df['vader_prediction']))


if __name__ == "__main__":

    analyze_finance_data()
```

LAB4(DATA):

```
import pandas as pd

from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer

from sklearn.metrics import classification_report


def analyze_general_sentiment_data():

    analyzer = SentimentIntensityAnalyzer()

    print("Loading General Sentiment Dataset...")

    try:

        df = pd.read_csv('lab5_sentiment_data.csv', encoding='latin-1').dropna()

    except FileNotFoundError:

        print("Error: lab5_sentiment_data.csv not found.")

        return

    # Map the numeric sentiment (Assuming 0: Negative, 1: Neutral, 2: Positive)

    sentiment_map = {0: 'Negative', 1: 'Neutral', 2: 'Positive'}

    df['true_label'] = df['Sentiment'].map(sentiment_map)

    def get_vader_label(text):

        score = analyzer.polarity_scores(str(text))['compound']

        if score >= 0.05: return "Positive"

        elif score <= -0.05: return "Negative"

        else: return "Neutral"


    print("Applying VADER Sentiment Analysis...")

    df['vader_prediction'] = df['text'].apply(get_vader_label)

    print("\n--- General Sentiment Data Results ---")

    print(df[['text', 'true_label', 'vader_prediction']].head(20))

    # Generate Classification Report

    print("\n--- Classification Report ---")

    valid_df = df.dropna(subset=['true_label', 'vader_prediction'])

    print(classification_report(valid_df['true_label'], valid_df['vader_prediction']))

if __name__ == "__main__":

    analyze_general_sentiment_data()
```

LAB5(twiiter):

```
import pandas as pd

from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer

from sklearn.metrics import classification_report


def analyze_twitter_data():

    analyzer = SentimentIntensityAnalyzer()

    print("Loading Twitter Dataset...")

    try:

        df = pd.read_csv('lab5_Twitter_Data_sentiment.csv', encoding='latin-1').dropna()

    except FileNotFoundError:

        print("Error: lab5_Twitter_Data_sentiment.csv not found.")

        return

    # Map Original Categories to Text

    category_map = {-1.0: 'Negative', 0.0: 'Neutral', 1.0: 'Positive'}

    df['true_label'] = df['category'].map(category_map)

    def get_vader_label(text):

        score = analyzer.polarity_scores(str(text))['compound']

        if score >= 0.05: return "Positive"

        elif score <= -0.05: return "Negative"

        else: return "Neutral"

    print("Applying VADER Sentiment Analysis...")

    df['vader_prediction'] = df['text'].apply(get_vader_label)

    print("\n--- Twitter Data Results ---")

    print(df[['text', 'true_label', 'vader_prediction']].head())

    # Generate Classification Report

    print("\n--- Classification Report ---")

    # Drop any rows where true_label might be NaN due to mapping issues

    valid_df = df.dropna(subset=['true_label', 'vader_prediction'])

    print(classification_report(valid_df['true_label'], valid_df['vader_prediction']))

if __name__ == "__main__":

    analyze_twitter_data()
```

LAB7(FB):

```python
import pandas as pd

import networkx as nx


def run_lab7_facebook():

    print("Loading Facebook Dataset...")

    df = pd.read_csv('lab8_facebook_edges.csv')


    # Create directed graph

    print("Building Directed Graph...")

    G = nx.from_pandas_edgelist(df, 'source', 'target', create_using=nx.DiGraph())

    print(f"Nodes: {G.number_of_nodes()} | Edges: {G.number_of_edges()}")


    # Apply PageRank

    print("Computing PageRank (alpha=0.85)...")

    pagerank = nx.pagerank(G, alpha=0.85)


    # Display ranking

    sorted_pr = sorted(pagerank.items(), key=lambda x: x[1], reverse=True)

    print("\n--- Top 10 Facebook Nodes by PageRank ---")

    for page, rank in sorted_pr[:10]:

        print(f"Node {page} : {round(rank, 6)}")


if __name__ == "__main__":

    run_lab7_facebook()
```

lab 7- PageRank(git)

```
import pandas as pd

import networkx as nx


def run_lab7_github():

    print("Loading GitHub Dataset...")

    df = pd.read_csv("C:/Users/KUNAL SHETIYA/Astma-LCA3/lab8_github_edges.csv")


    print("Building Directed Graph...")

    G = nx.from_pandas_edgelist(

        df,

        'source',

        'target',

        create_using=nx.DiGraph()

    )


    print(f"Nodes: {G.number_of_nodes()} | Edges: {G.number_of_edges()}")


    print("Computing PageRank (alpha=0.85)...")

    pagerank = nx.pagerank(G, alpha=0.85)


    sorted_pr = sorted(pagerank.items(), key=lambda x: x[1], reverse=True)


    print("\n--- Top 10 GitHub Nodes by PageRank ---")

    for i, (page, rank) in enumerate(sorted_pr[:10]):

        print(f"Rank {i+1}: Node {page} = {rank:.6f}")


if __name__ == "__main__":

    run_lab7_github()
```

Lab7-pagerank(higgs)-

```
import pandas as pd

import networkx as nx


def run_lab7_higgs():

    print("Loading Higgs Twitter Dataset...")

    df = pd.read_csv("C:/Users/KUNAL SHETIYA/Astma-LCA3/lab8_higgs-social_network.csv")


    print("Building Directed Graph...")

    G = nx.from_pandas_edgelist(

        df,

        'source',

        'target',

        create_using=nx.DiGraph()

    )


    print(f"Nodes: {G.number_of_nodes()} | Edges: {G.number_of_edges()}")


    print("Computing PageRank (alpha=0.85)...")

    pagerank = nx.pagerank(G, alpha=0.85)


    sorted_pr = sorted(pagerank.items(), key=lambda x: x[1], reverse=True)


    print("\n--- Top 10 Higgs Nodes by PageRank ---")

    for i, (page, rank) in enumerate(sorted_pr[:10]):

        print(f"Rank {i+1}: Node {page} = {rank:.6f}")


if __name__ == "__main__":

    run_lab7_higgs()
```

lab8(fb):

```python
import pandas as pd
import networkx as nx

def run_lab8_facebook():
    print("Loading Facebook Dataset...")
    df = pd.read_csv('lab8_facebook_edges.csv')

    # Create directed graph
    print("Building Directed Graph...")
    G = nx.from_pandas_edgelist(df, 'source', 'target', create_using=nx.DiGraph())

    # Compute Metrics
    print("Computing Degree Centrality...")
    degree = nx.degree_centrality(G)

    print("Computing Betweenness Centrality (Approximation k=100)...")
    betweenness = nx.betweenness_centrality(G, k=100, seed=42)

    # Sort and Display Top 5
    top_degree = sorted(degree.items(), key=lambda x: x[1], reverse=True)[:5]
    top_between = sorted(betweenness.items(), key=lambda x: x[1], reverse=True)[:5]

    print("\n--- Facebook Top 5: Degree Centrality (Most Connected) ---")
    for node, val in top_degree: print(f"Node {node}: {val:.5f}")

    print("\n--- Facebook Top 5: Betweenness Centrality (Best Connectors) ---")
    for node, val in top_between: print(f"Node {node}: {val:.5f}")

if __name__ == "__main__":
    run_lab8_facebook()
```

Lab8- Centrality_git –

```
import pandas as pd

import networkx as nx

def run_lab8_github():

    print("Loading GitHub Dataset...")

    df = pd.read_csv("C:/Users/KUNAL SHETIYA/Astma-LCA3/lab8_github_edges.csv")


    # Create directed graph

    print("Building Directed Graph...")

    G = nx.from_pandas_edgelist(df, 'source', 'target', create_using=nx.DiGraph())


    # Compute Metrics

    print("Computing Degree Centrality...")

    degree = nx.degree_centrality(G)


    print("Computing Betweenness Centrality (Approximation k=100)...")

    betweenness = nx.betweenness_centrality(G, k=100, seed=42)


    # Sort and Display Top 5

    top_degree = sorted(degree.items(), key=lambda x: x[1], reverse=True)[:5]

    top_between = sorted(betweenness.items(), key=lambda x: x[1], reverse=True)[:5]


    print("\n--- GitHub Top 5: Degree Centrality (Most Connected) ---")

    for node, val in top_degree: print(f"Node {node}: {val:.5f}")


    print("\n--- GitHub Top 5: Betweenness Centrality (Best Connectors) ---")

    for node, val in top_between: print(f"Node {node}: {val:.5f}")


if __name__ == "__main__":

    run_lab8_github()
```

lab8 – centrality(higgs) –

```
import pandas as pd

import networkx as nx

def run_lab8_higgs():

    print("Loading Higgs Dataset...")

    df = pd.read_csv("C:/Users/KUNAL SHETIYA/Astma-LCA3/lab8_higgs-social_network.csv")


    # Create directed graph

    print("Building Directed Graph...")

    G = nx.from_pandas_edgelist(df, 'source', 'target', create_using=nx.DiGraph())


    # Compute Metrics

    print("Computing Degree Centrality...")

    degree = nx.degree_centrality(G)


    print("Computing Betweenness Centrality (Approximation k=100)...")

    betweenness = nx.betweenness_centrality(G, k=100, seed=42)


    # Sort and Display Top 5

    top_degree = sorted(degree.items(), key=lambda x: x[1], reverse=True)[:5]

    top_between = sorted(betweenness.items(), key=lambda x: x[1], reverse=True)[:5]


    print("\n--- Higgs Top 5: Degree Centrality (Most Connected) ---")

    for node, val in top_degree: print(f"Node {node}: {val:.5f}")


    print("\n--- Higgs Top 5: Betweenness Centrality (Best Connectors) ---")

    for node, val in top_between: print(f"Node {node}: {val:.5f}")


if __name__ == "__main__":

    run_lab8_higgs()
```